

Aula 9 - Spring Data JPA + MySQL (CRUD Completo)

Duração: 4 horas (1h teórica + 3h prática)

Parte Teórica (1 hora)

1. Revisão da Aula Anterior (15 min)

- Discussão sobre:
 - Arquitetura Controller-Service.
 - Dúvidas sobre injeção de dependências (`@Autowired`).

2. Introdução ao Spring Data JPA (25 min)

Conceitos Fundamentais

- JPA (Java Persistence API): Especificação para mapeamento objeto-relacional (ORM).
- Spring Data JPA: Camada de abstração sobre JPA para operações CRUD.

Anotações Principais

Anotação	Uso
<code>@Entity</code>	Define uma classe como entidade do BD
<code>@Id</code>	Indica a chave primária
<code>@GeneratedValue</code>	Configura autoincremento
<code>@Repository</code>	Interface que estende <code>JpaRepository</code>

Métodos Automáticos

```
java

public interface ProductRepository extends JpaRepository<Product, Long> {
    // Métodos herdados: save(), findAll(), findById(), deleteById()
}
```

3. Configuração do MySQL (20 min)

1. Adicionar dependências no pom.xml :

xml

```
<dependency>
  <groupId>mysql</groupId>
  <artifactId>mysql-connector-java</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

2. Configurar application.properties :

properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/spring_db
spring.datasource.username=root
spring.datasource.password=senha123
spring.jpa.hibernate.ddl-auto=update
```

Parte Prática (3 horas)

Atividade: CRUD Completo com MySQL

Passo 1: Modelar Entidade (30 min)

1. Classe Product (model/Product.java):

java

```
@Entity
@Getter @Setter @NoArgsConstructor
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @Column(nullable = false)
    private double price;
}
```

2. Criar banco de dados manualmente:

sql

```
CREATE DATABASE spring_db;
```

Passo 2: Implementar Repository (45 min)

1. Interface `ProductRepository`:

java

```
@Repository
public interface ProductRepository extends JpaRepository<Product, Long> {
    // Consulta customizada (exemplo)
    List<Product> findByPriceLessThan(double price);
}
```

2. Atualizar `ProductService`:

java

```
@Service
public class ProductService {
    @Autowired
    private ProductRepository repository;

    public List<Product> getAll() {
        return repository.findAll();
    }

    public Product create(Product product) {
        return repository.save(product);
    }
}
```

Passo 3: Testar Endpoints com Postman (1h)

1. Coleção de requisições:

Método	Endpoint	Body (JSON)
POST	/products	{"name": "Mouse", "price": 50.0}
GET	/products	-
GET	/products/1	-
PUT	/products/1	{"name": "Teclado", "price": 120}

Método	Endpoint	Body (JSON)
DELETE	/products/1	-

2. Implemente PUT e DELETE no ProductService :

```
java

public Product update(Long id, Product product) {
    product.setId(id);
    return repository.save(product);
}

public void delete(Long id) {
    repository.deleteById(id);
}
```

Passo 4: Validações Básicas (45 min)

1. Adicione validações na entidade:

```
java

@Column(nullable = false, length = 100)
@NotBlank(message = "Nome é obrigatório")
private String name;

@Positive(message = "Preço deve ser positivo")
private double price;
```

2. Trate erros no Controller:

```
java

@PostMapping
public ResponseEntity<Product> create(@Valid @RequestBody Product product) {
    return ResponseEntity.status(201).body(service.create(product));
}
```

Tarefa para Casa

1. Consultas customizadas:

- o Implemente um endpoint /products/cheaper-than?maxPrice=100 .

2. Tratamento de erros:

- Capture `DataIntegrityViolationException` e retorne HTTP 400.

Próxima Aula

- **Tópico:** Relacionamentos JPA (OneToMany, ManyToMany).
- **Atividade prática:** Sistema de pedidos com clientes e produtos.

Resultado Esperado

- API com CRUD completo persistido no MySQL.
- Validações de campos e tratamento básico de erros.

plaintext

- ✓ Endpoints testados
- ✓ Dados persistidos no banco
- ✓ Pronto para relacionamentos entre tabelas

<https://www.baeldung.com/wp-content/uploads/2018/11/spring-data.png> (Spring Data JPA em ação) 🚀